

comprehensive study plan: from bundlers to runtime internals

navigation guide

this document contains 4 major learning paths:

1. bundler internals (esbuild)
2. fastify internals
3. node.js internals
4. bun internals

each path contains:

- **prerequisites:** what you need before starting
- **core concepts:** what you must understand
- **source code:** specific files to read
- **build projects:** what to implement
- **verification:** how to confirm understanding
- **resources:** where to learn

progress sequentially. don't skip verification checkpoints.

PART 1: BUNDLER INTERNALS

phase 1.1: fundamentals

prerequisites

- ☐ basic understanding of imports/exports in JavaScript
- ☐ node.js installed
- ☐ git, code editor

core concepts

module systems

- CommonJS (require/module.exports)
- ESM (import/export)
- AMD (define/require)
- why multiple systems exist
- interoperability challenges

dependency graphs

- nodes = modules (files)
- edges = imports between files
- entry points
- circular dependencies
- traversal algorithms

abstract syntax trees (AST)

- code as data structures
- tokenization (lexing)
- parsing (syntax analysis)
- tree representation of code
- why bundlers need ASTs

module resolution

- relative paths (/file.js)
- absolute paths (/root/file.js)
- node_modules lookup algorithm
- package.json main/exports fields
- file extensions (.js, .ts, .jsx)
- index.js convention

source code to read

node.js module resolution:

- <https://nodejs.org/api/esm.html>
- focus on: resolution algorithm section
- understand: how node finds files

webpack documentation:

- <https://webpack.js.org/concepts/module-resolution/>
- focus on: resolution process
- understand: configurable resolution

build project: module resolver

requirements:

javascript

// must handle:

`resolve('./utils.js', '/project/src/index.js')`

// → /project/src/utils.js

`resolve('./utils', '/project/src/index.js')`

// → /project/src/utils.js (try extensions)

`resolve('lodash', '/project/src/index.js')`

// → /project/node_modules/lodash/index.js

`resolve('lodash/get', '/project/src/index.js')`

// → /project/node_modules/lodash/get.js

implementation steps:

1. handle relative paths using `path.resolve()`
2. check file extensions: `.js`, `.ts`, `.jsx`, `.tsx`, `.json`
3. check `index.js` in directories
4. search `node_modules` recursively upward
5. read `package.json` for `main/exports`
6. handle errors (file not found)

file structure:

resolver.js (your implementation)

test.js (test cases)

fixtures/ (test files)

verification checkpoint

you understand module resolution when you can:

- ☐ explain why node_modules search goes upward
 - ☐ trace resolution manually for any import
 - ☐ implement working resolver (100-150 lines)
 - ☐ handle edge cases (circular symlinks, missing files)
 - ☐ write blog post explaining resolution algorithm
-

phase 1.2: parsing and AST

prerequisites

- ☐ completed phase 1.1
- ☐ understands module resolution

core concepts

lexical analysis (tokenization)

- scanner/lexer role
- token types (identifier, keyword, operator, literal)
- whitespace handling
- comments

syntax analysis (parsing)

- parser role
- recursive descent parsing
- operator precedence
- grammar rules
- syntax errors

AST structure

- program node (root)
- statement nodes (if, for, function, etc)
- expression nodes (binary, call, member, etc)
- declaration nodes (variable, function)
- import/export nodes

AST manipulation

- traversal (visiting nodes)
- transformation (modifying nodes)
- code generation (AST $\rightarrow$ code)

source code to read

tiny compiler:

- <https://github.com/jamiebuilds/the-super-tiny-compiler>
- read entire repo (200 lines)
- understand: tokenizer $\rightarrow$ parser $\rightarrow$ transformer $\rightarrow$ generator

babel parser basics:

- @babel/parser documentation
- focus on: AST node types
- use astexplorer.net to visualize

build project: import/export extractor

requirements:

javascript

// input:

```
const code = `
import React from 'react';
import { useState, useEffect } from 'react';
import * as utils from './utils';
export const name = 'Chetan';
export default App;
`;

// output:
{
  imports: [
    { source: 'react', specifiers: ['default as React'] },
    { source: 'react', specifiers: ['useState', 'useEffect'] },
    { source: './utils', specifiers: ['* as utils'] }
  ],
  exports: [
    { type: 'named', name: 'name' },
    { type: 'default', name: 'App' }
  ]
}
```

implementation:

1. parse code using @babel/parser
2. traverse AST using @babel/traverse
3. find ImportDeclaration nodes
4. find ExportNamedDeclaration nodes
5. find ExportDefaultDeclaration nodes
6. extract source and specifiers
7. return structured data

verification checkpoint

you understand parsing when you can:

- ☐ explain difference between lexer and parser
- ☐ manually draw AST for simple code
- ☐ use @babel/parser and @babel/traverse
- ☐ extract all imports/exports from any file
- ☐ write blog post explaining ASTs

phase 1.3: esbuild architecture

prerequisites

- ☐ completed phases 1.1 and 1.2
- ☐ go installed
- ☐ basic go syntax (structs, functions, pointers)

core concepts

esbuild pipeline:

1. scanning: find all files
2. parsing: code $\rightarrow$ AST
3. linking: resolve imports, build graph
4. bundling: combine files
5. code generation: output bundles

why esbuild is fast:

- written in go (compiled, not interpreted)
- parallelization (goroutines)
- efficient memory usage
- minimal abstractions
- optimized algorithms

go concurrency basics:

- goroutines (lightweight threads)
- channels (communication between goroutines)
- sync primitives (mutexes, waitgroups)
- why this matters for bundler speed

source code to read

esbuild architecture:

- <https://github.com/evanw/esbuild>
- read: /docs/architecture.md (multiple times)
- understand: high-level flow

key go files:

- /internal/bundler/bundler.go (main bundling logic)
- /internal/js_parser/js_parser.go (parser)
- /internal/resolver/resolver.go (module resolution)
- /internal/graph/graph.go (dependency graph)

reading approach:

1. start with bundler.go entry point
2. follow function calls
3. understand data structures (structs)
4. note use of goroutines
5. don't understand everything - 40% is fine
6. return to confusing parts later

go language crash course

learn only what you need:

structs:

```
go

type Module struct {
    path string
    imports []string
}
```

pointers:

```
go

func resolve(m *Module) { // pointer to Module
    m.path = "/new/path" // modifies original
}
```

methods:

```
go

func (r *Resolver) Resolve(path string) string {
    // r is receiver
    return r.basePath + path
}
```


slices (dynamic arrays):

```
go

modules := make([]Module, 0)
modules = append(modules, Module{path: "index.js"})
```

maps:

```
go

cache := make(map[string]string)
cache["key"] = "value"
```

resources:

- tour of go: <https://go.dev/tour/>
- read sections: basics, methods, concurrency
- skip: interfaces (for now)
- reference "the go programming language" book when stuck

build project: minimal bundler

requirements:

- entry point: index.js
- resolves imports (use your resolver)
- parses files (use @babel/parser)
- builds dependency graph
- outputs single bundle

algorithm:

javascript

```
function bundle(entryPath) {
  const modules = new Map(); // path → module info
  const queue = [entryPath];

  while (queue.length > 0) {
    const currentPath = queue.shift();

    // skip if already processed
    if (modules.has(currentPath)) continue;

    // read file
    const code = fs.readFileSync(currentPath, 'utf8');

    // parse and extract imports
    const imports = extractImports(code);

    // resolve each import
    const resolvedImports = imports.map(imp =>
      resolve(imp, currentPath)
    );

    // store module
    modules.set(currentPath, {
      code,
      imports: resolvedImports
    });

    // add imports to queue
    queue.push(...resolvedImports);
  }

  // generate output
  return generateBundle(modules, entryPath);
}
```

output format (simple):

javascript

```
(function() {  
  const modules = {  
    'index.js': function(exports, require) {  
      // transformed code here  
    },  
    'utils.js': function(exports, require) {  
      // transformed code here  
    }  
  };  
  
  function require(path) {  
    const module = { exports: {} };  
    modules[path](module.exports, require);  
    return module.exports;  
  }  
  
  require('index.js'); // start execution  
})();
```

verification checkpoint

you understand bundlers when you can:

- ☐ explain esbuild's pipeline stages
- ☐ read go code in esbuild repo (understand 40%+)
- ☐ build working bundler that handles multiple files
- ☐ handle circular dependencies
- ☐ generate executable output
- ☐ write blog post: "how bundlers work"

phase 1.4: advanced bundler concepts

prerequisites

- ☐ completed phase 1.3
- ☐ have working bundler

core concepts

tree shaking (dead code elimination)

- static analysis of exports
- tracking used vs unused exports
- requires ESM (not CommonJS)
- side effects consideration
- why it matters for bundle size

code splitting

- multiple entry points
- dynamic imports (import())
- shared dependencies
- chunk generation
- lazy loading strategies

optimization techniques

- minification (remove whitespace, shorten names)
- scope hoisting (concatenate modules)
- constant folding (evaluate at build time)
- dead code elimination
- deduplication

source maps

- debugging bundled code
- mapping generated → original positions
- VLQ encoding
- source map format
- inline vs external maps

caching and incremental builds

- file-level caching
- dependency tracking
- invalidation strategies
- persistent caching
- why esbuild is fast on rebuilds

source code to read

esbuild tree shaking:

- /internal/bundler/linker.go
- search for: "tree shaking", "dead code"
- understand: symbol tracking

esbuild code splitting:

- /internal/bundler/bundler.go
- search for: "splitting", "chunks"
- understand: chunk generation

esbuild minification:

- /internal/js_printer/js_printer.go
- /internal/renamer/renamer.go
- understand: identifier mangling

source maps:

- /internal/sourcemap/sourcemap.go
- source map spec: <https://sourcemap.info/spec.html>

build projects

project 1: add tree shaking

- track which exports are imported
- remove unused exports from bundle
- handle side effects correctly
- test with: lodash (large library)

project 2: add minification

- remove whitespace and comments
- shorten variable names
- implement name mangling (a, b, c...)
- preserve semantics

project 3: add source maps

- generate basic source map
- map bundled positions → original
- test in browser devtools
- verify breakpoints work

verification checkpoint

you understand advanced bundling when you can:

- ☐ explain why tree shaking needs ESM
 - ☐ implement basic tree shaking
 - ☐ explain code splitting strategies
 - ☐ implement minification
 - ☐ generate source maps
 - ☐ explain esbuild's caching strategy
 - ☐ write blog post: "advanced bundler techniques"
-

phase 1.5: real-world concerns

core concepts

performance optimization

- parallelization strategies
- memory management
- I/O optimization
- caching layers
- profiling and benchmarking

error handling

- parse errors
- resolution errors
- circular dependencies
- missing files
- helpful error messages

plugin systems

- transform hooks
- resolve hooks
- load hooks
- plugin API design
- esbuild vs webpack plugins

configuration

- entry points
- output options
- loader configuration
- target environments
- mode (development vs production)

source code to read

esbuild parallelization:

- `/internal/bundler/bundler.go`
- search for: goroutines, channels, sync
- understand: parallel parsing

esbuild error handling:

- `/internal/logger/logger.go`
- observe: error message formatting

build project: production bundler

add to your bundler:

- configuration file support
- multiple entry points
- watch mode (rebuild on changes)
- error reporting with source locations
- basic plugin system
- benchmarking output

comparison test:

```
bash

# bundle same project with:
your-bundler index.js -o dist/
esbuild index.js --bundle --outfile=dist/esbuild.js
webpack index.js -o dist/

# compare:
- build time
- bundle size
- correctness
```

verification checkpoint

you mastered bundlers when you can:

- ☐ build production-ready bundler
- ☐ understand esbuild source code (60%+)
- ☐ explain performance optimizations
- ☐ debug bundler issues at work
- ☐ contribute to webpack/esbuild discussions
- ☐ write series: "building a bundler from scratch"

PART 2: FASTIFY INTERNALS

phase 2.1: http server basics

prerequisites

- ☐ completed bundler internals
- ☐ understand async/await
- ☐ basic node.js knowledge

core concepts

node.js http module

- createServer()
- request/response objects
- headers, status codes, body
- streaming responses
- connection handling

middleware pattern

- request processing pipeline
- next() function
- error handling
- middleware composition
- execution order

routing

- URL parsing
- path matching (exact, parameterized, wildcard)
- route precedence
- HTTP methods (GET, POST, etc)
- route organization

source code to read

node.js http:

- node source: /lib/http.js (skim)
- node docs: <https://nodejs.org/api/http.html>
- understand: basic server creation

express basics:

- <https://github.com/expressjs/express>
- /lib/router/index.js (routing)
- understand: middleware pattern

build project: basic http server

requirements:

```
javascript

const server = createServer();

server.get('/users/:id', (req, res) => {
  res.json({ id: req.params.id });
});

server.post('/users', (req, res) => {
  // handle body
  res.status(201).json({ created: true });
});

server.listen(3000);
```

features:

- route registration (get, post, put, delete)
- path parameters (:id)
- response helpers (json, status)
- request body parsing
- error handling

verification checkpoint

you understand http servers when you can:

- ☐ build basic http server from scratch
- ☐ implement routing without libraries
- ☐ explain middleware pattern
- ☐ handle different content types
- ☐ write blog post: "how http servers work"

phase 2.2: fastify architecture

prerequisites

- ☐ completed phase 2.1
- ☐ understand decorators pattern
- ☐ schema validation concepts

core concepts

fastify design principles

- schema-based validation
- plugin architecture
- encapsulation
- decorators
- lifecycle hooks
- why it's faster than express

request/response lifecycle

1. onRequest hook
2. preParsing hook
3. preValidation hook
4. preHandler hook
5. handler execution
6. preSerialization hook
7. onSend hook
8. onResponse hook

plugin system

- encapsulation boundaries
- plugin registration
- dependency injection
- decorators
- hooks

schema compilation

- JSON schema → validation function
- compile-time optimization
- ajv integration
- serialization schemas
- why pre-compilation is fast

source code to read

fastify core:

- <https://github.com/fastify/fastify>
- /lib/route.js (route handling)
- /lib/reply.js (response object)
- /lib/request.js (request object)
- /lib/server.js (server setup)

focus on:

- route registration
- schema compilation
- hook execution
- plugin loading

build projects

project 1: understand fastify routing

javascript

// trace this code in fastify source:

```
fastify.get('/users/:id', {  
  schema: {  
    params: {  
      type: 'object',  
      properties: {  
        id: { type: 'string' }  
      }  
    }  
  }, async (request, reply) => {  
    return { id: request.params.id };  
  });
```

// understand:

// - how schema gets compiled

// - how route gets stored

// - how handler gets wrapped

// - how validation happens

project 2: implement mini-fastify

- plugin system with encapsulation
- schema validation (use ajv)
- lifecycle hooks
- decorators
- route registration

verification checkpoint

you understand fastify when you can:

- ☐ explain why fastify is fast
 - ☐ trace request lifecycle in source
 - ☐ implement plugin system
 - ☐ use schema compilation for validation
 - ☐ explain encapsulation boundaries
 - ☐ write blog post: "fastify internals"
-

phase 2.3: performance optimization

prerequisites

- ☐ completed phase 2.2
- ☐ basic understanding of V8

core concepts

fastify performance techniques

- schema compilation (ajv)
- find-my-way router (radix tree)
- fast-json-stringify serialization
- object pooling
- minimal abstraction

benchmarking

- autocannon for load testing
- clinic.js for profiling
- flamegraphs
- memory profiling
- identifying bottlenecks

optimization patterns

- avoid dynamic property access
- monomorphic functions
- hidden classes
- inline caches
- memory allocation patterns

source code to read

find-my-way (router):

- <https://github.com/delvedor/find-my-way>
- understand: radix tree routing
- compare: vs linear search

fast-json-stringify:

- <https://github.com/fastify/fast-json-stringify>
- understand: why schema-based serialization is fast

fastify benchmarks:

- /benchmarks/ directory
- understand: what they measure

build project: benchmark your server

compare:

```
javascript
```

```
// your basic server vs fastify vs express
```

```
// benchmark:
```

```
- requests per second
```

```
- latency (p50, p95, p99)
```

```
- memory usage
```

```
- CPU usage
```

```
// use autocannon:
```

```
autocannon -c 100 -d 10 http://localhost:3000/users/123
```

profile and optimize:

1. identify bottleneck
2. optimize
3. measure improvement
4. repeat

verification checkpoint

you understand performance when you can:

- ☐ explain fastify's optimization techniques
- ☐ benchmark servers accurately
- ☐ profile and find bottlenecks
- ☐ optimize request handlers
- ☐ achieve 10k+ RPS on simple endpoint
- ☐ write blog post: "fastify performance deep dive"

PART 3: NODE.JS INTERNALS

phase 3.1: event loop fundamentals

prerequisites

- ☐ completed bundlers or fastify
- ☐ understand async JavaScript
- ☐ C basics (helpful but not required)

core concepts

event loop phases

1. timers (setTimeout, setInterval)
2. pending callbacks (I/O callbacks)
3. idle, prepare (internal)
4. poll (retrieve I/O events)
5. check (setImmediate)
6. close callbacks

microtasks vs macrotasks

- process.nextTick (highest priority)
- Promise callbacks (microtasks)
- timers, I/O (macrotasks)
- execution order
- why order matters

libuv architecture

- event loop implementation
- thread pool for blocking operations
- platform-specific I/O (epoll, kqueue, IOCP)
- timers implementation
- handle types

source code to read

node.js event loop:

- <https://nodejs.org/en/docs/guides/event-loop-timers-and-nexttick/>
- read multiple times until clear

libuv:

- <https://github.com/libuv/libuv>
- /docs/design.rst (architecture overview)
- /src/unix/core.c (event loop implementation)
- uv_run() function

node integration:

- <https://github.com/nodejs/node>
- /src/node_main_instance.cc (bootstrap)
- /lib/internal/bootstrap/node.js

build project: event loop visualization

implement:

```
javascript

// trace execution order:
console.log('1: sync');

setTimeout(() => console.log('2: timer'), 0);

Promise.resolve().then(() => console.log('3: promise'));

process.nextTick(() => console.log('4: nextTick'));

setImmediate(() => console.log('5: immediate'));

fs.readFile('file.txt', () => {
  console.log('6: I/O callback');

  setTimeout(() => console.log('7: timer in I/O'), 0);

  setImmediate(() => console.log('8: immediate in I/O'));

  process.nextTick(() => console.log('9: nextTick in I/O'));
});

console.log('10: sync end');

// predict order before running
// run and verify
// explain why
```

build visualizer:

- logs event loop phases
- tracks microtask queue
- tracks macrotask queue
- shows execution order

verification checkpoint

you understand event loop when you can:

- ☐ predict execution order of async code
 - ☐ explain all event loop phases
 - ☐ explain microtask vs macrotask
 - ☐ read libuv source code (basics)
 - ☐ debug async timing issues
 - ☐ write blog post: "node.js event loop explained"
-

phase 3.2: streams and buffers

prerequisites

- ☐ completed phase 3.1
- ☐ understand binary data basics

core concepts

streams types

- Readable (fs.createReadStream)
- Writable (fs.createWriteStream)
- Duplex (net.Socket)
- Transform (zlib.createGzip)

stream mechanics

- backpressure
- piping
- flowing vs paused mode
- highWaterMark
- events (data, end, error, finish)

buffers

- fixed-size memory allocation
- binary data manipulation
- encoding/decoding (UTF-8, base64, hex)
- performance considerations
- Buffer vs Uint8Array

practical streaming

- large file processing
- HTTP request/response streaming
- compression/decompression
- parsing streams
- memory efficiency

source code to read

node.js streams:

- /lib/stream.js (readable, writable)
- /lib/internal/streams/*.js
- focus on: backpressure implementation

buffer implementation:

- /lib/buffer.js
- /src/node_buffer.cc (C++ internals)

build projects

project 1: stream processor

```
javascript

// process large file (1GB+) with constant memory
// transform: CSV → JSON
// requirements:
// - streaming read
// - streaming parse
// - streaming write
// - handle backpressure
// - memory usage < 100MB
```

project 2: implement custom transform stream

- extends Transform class
- processes chunks
- handles encoding
- proper error handling

verification checkpoint

you understand streams when you can:

- ☐ process files larger than RAM
 - ☐ implement custom stream types
 - ☐ explain backpressure mechanism
 - ☐ handle binary data with buffers
 - ☐ optimize memory usage
 - ☐ write blog post: "mastering node.js streams"
-

phase 3.3: V8 integration

prerequisites

- ☐ completed phases 3.1 and 3.2
- ☐ basic C++ (helpful)

core concepts

V8 in node.js

- embedding V8 engine
- isolates and contexts
- handle scope management
- native bindings
- addon architecture

native modules

- N-API (node addon API)
- creating C++ addons
- exposing functions to JavaScript
- memory management
- performance considerations

garbage collection

- generational GC
- minor GC (scavenge)
- major GC (mark-sweep-compact)
- incremental marking
- GC triggers and tuning

performance implications

- JIT compilation
- optimization tiers
- deoptimization
- hidden classes
- inline caches

source code to read

node.js V8 bindings:

- /src/node_main.cc (V8 initialization)
- /src/node_buffer.cc (native module example)
- /src/node_api.cc (N-API implementation)

V8 documentation:

- <https://v8.dev/docs>
- embedder's guide
- GC documentation

build project: native addon

create addon that:

- exposes C++ function to JavaScript
- performs CPU-intensive task
- benchmark: pure JS vs native
- handles errors properly
- uses N-API (stable ABI)

example:

```
cpp

// addon.cc

#include <napi.h>

Napi::Number ProcessData(const Napi::CallbackInfo& info) {
    // intensive computation
    return Napi::Number::New(info.Env(), result);
}

Napi::Object Init(Napi::Env env, Napi::Object exports) {
    exports.Set("processData", Napi::Function::New(env, ProcessData));
    return exports;
}

NODE_API_MODULE(addon, Init)
```

verification checkpoint

you understand V8 integration when you can:

- ☐ explain how node embeds V8
 - ☐ create native addons
 - ☐ understand GC behavior
 - ☐ profile and optimize V8 usage
 - ☐ explain optimization/deoptimization
 - ☐ write blog post: "V8 in node.js"
-

phase 3.4: networking internals

prerequisites

- ☐ completed phase 3.1
- ☐ understand TCP/IP basics

core concepts

TCP server implementation

- `net.createServer()`
- socket handling
- connection events
- data parsing
- error handling
- keepalive, timeout

HTTP implementation on TCP

- parsing HTTP protocol
- header parsing
- body handling
- chunked encoding
- HTTP/1.1 features
- HTTP/2 differences

TLS/SSL

- secure sockets
- certificate handling
- handshake process
- performance impact

DNS resolution

- `dns.lookup()` vs `dns.resolve()`
- caching
- custom resolvers

source code to read

node.js networking:

- `/lib/net.js` (TCP implementation)
- `/lib/_http_server.js` (HTTP server)
- `/lib/_http_client.js` (HTTP client)
- `/lib/https.js` (HTTPS)

libuv networking:

- /src/unix/tcp.c (TCP implementation)
- /src/unix/stream.c (stream sockets)

build project: HTTP server from TCP

build HTTP/1.1 server using only net module:

```
javascript

const net = require('net');

const server = net.createServer(socket => {
  socket.on('data', data => {
    // parse HTTP request
    // extract method, path, headers, body
    // generate HTTP response
    // write response to socket
  });
});

// must handle:
// - request parsing
// - response formatting
// - multiple requests on same connection
// - proper connection closing
```

verification checkpoint

you understand networking when you can:

- ☐ build HTTP server from TCP sockets
- ☐ parse HTTP protocol manually
- ☐ explain connection handling
- ☐ optimize socket performance
- ☐ debug network issues
- ☐ write blog post: "HTTP from scratch in node.js"

phase 3.5: cluster and worker threads

prerequisites

- ☐ completed previous node.js phases
- ☐ understand process vs thread

core concepts

cluster module

- master/worker processes
- load balancing
- IPC (inter-process communication)
- zero-downtime restart
- shared server ports

worker threads

- thread pool
- SharedArrayBuffer
- MessageChannel
- transferable objects
- when to use threads

comparison: cluster vs workers

- process isolation vs shared memory
- CPU-bound vs I/O-bound tasks
- overhead comparison
- use cases

source code to read

cluster:

- /lib/cluster.js (cluster API)
- /lib/internal/cluster/master.js
- /lib/internal/cluster/child.js

worker threads:

- /lib/worker_threads.js
- /src/node_worker.cc (C++ implementation)

build project: production server

implement:

- multi-process server using cluster
- graceful shutdown
- zero-downtime restart
- health checks
- CPU-intensive task offloading to workers

compare:

```
bash
```

```
# benchmark:
```

```
single process vs cluster(4) vs worker threads
```

```
measure: throughput, latency, CPU usage
```

verification checkpoint

you understand concurrency when you can:

- ☐ implement multi-process architecture
- ☐ use worker threads effectively
- ☐ explain process vs thread tradeoffs
- ☐ handle graceful shutdown
- ☐ optimize for multi-core systems
- ☐ write blog post: "scaling node.js applications"

PART 4: BUN INTERNALS

phase 4.1: bun vs node architecture

prerequisites

- ☐ completed node.js internals
- ☐ understand JavaScriptCore basics
- ☐ basic zig syntax (optional)

core concepts

architectural differences

- V8 vs JavaScriptCore
- C++ vs zig
- libuv vs custom event loop
- npm vs bun package manager
- bundler integration

performance improvements

- faster startup time
- faster module resolution
- faster require/import
- faster HTTP server
- integrated bundler/transpiler

compatibility

- node.js API compatibility
- npm package compatibility
- limitations and differences
- migration considerations

source code to read

bun:

- <https://github.com/oven-sh/bun>
- /docs/runtime.md (architecture)
- /src/bun.js/bindings/ (JS bindings)
- /src/bun.js/modules/ (built-in modules)

JavaScriptCore:

- <https://github.com/webkit/webkit>
- /Source/JavaScriptCore/
- focus on: basic architecture

build project: benchmark comparison

compare node vs bun:

bash

startup time

`time node -e "console.log('hello')"`

`time bun -e "console.log('hello')"`

module loading

(load 1000 modules)

HTTP server

(simple hello world endpoint)

bundling

(same project with webpack vs bun)

document results

explain why differences exist

verification checkpoint

you understand bun when you can:

- ☐ explain architectural differences vs node
 - ☐ benchmark accurately
 - ☐ explain performance improvements
 - ☐ understand compatibility implications
 - ☐ write blog post: "bun vs node: internals comparison"
-

phase 4.2: JavaScriptCore internals

prerequisites

- ☐ completed phase 4.1
- ☐ understand V8 basics (from node section)

core concepts

JSC architecture

- bytecode interpreter (LLInt)
- baseline JIT
- DFG (data flow graph) JIT
- FTL (faster than light) JIT
- tier-up strategy

differences from V8

- 4-tier vs 2-tier compilation
- different optimization strategies
- different GC implementation
- memory characteristics

practical implications

- when JSC is faster
- when V8 is faster
- optimization patterns
- benchmarking considerations

source code to read

JavaScriptCore:

- /Source/JavaScriptCore/jit/ (JIT implementations)
- /Source/JavaScriptCore/runtime/ (runtime)
- webkit blog posts on JSC

comparison:

- read V8 architecture docs
- compare with JSC architecture
- note differences

verification checkpoint

you understand JSC when you can:

- ☐ explain JSC compilation tiers
- ☐ compare JSC vs V8 architecture
- ☐ explain performance characteristics
- ☐ understand bun's engine choice
- ☐ write blog post: "JavaScriptCore explained"

phase 4.3: zig for JavaScript runtime

prerequisites

- ☐ completed phase 4.1
- ☐ basic systems programming concepts

core concepts

why zig for bun

- manual memory management
- compile-time execution
- C interop
- performance
- simplicity

zig basics for bun

- comptime
- error handling
- allocators
- slices and arrays
- pointers
- structs

bun's zig usage

- HTTP server implementation
- bundler implementation
- transpiler
- file system operations
- native modules

source code to read

zig language:

- <https://ziglang.org/learn/>
- focus on: basics, memory management

bun's zig code:

- /src/bun.zig (entry point)
- /src/http/ (HTTP server)
- /src/bundler/ (bundler)

build project: simple zig program

implement:

```
zig

// HTTP server in zig
// requirements:
// - handle connections
// - parse HTTP
// - respond
// - compare with node/bun performance
```

verification checkpoint

you understand zig in runtime context when you can:

- ☐ read bun's zig code (basics)
- ☐ explain memory management differences
- ☐ understand performance benefits
- ☐ write simple zig programs
- ☐ write blog post: "why bun uses zig"

PART 5: OPTIONAL EXTENSIONS

extension 5.1: rust for tooling

when to learn

- after completing bundlers
- when hitting JavaScript performance limits
- when interested in tools like swc, deno

core concepts

- ownership and borrowing
- zero-cost abstractions
- FFI (foreign function interface)
- N-API bindings
- WASM compilation

resources

- "the rust programming language" book
 - swc source code
 - deno source code
-

extension 5.2: system design

when to learn

- after implementing real systems
- when you understand distributed systems concepts
- when preparing for interviews

core concepts

- scalability patterns
- caching strategies
- database sharding
- load balancing
- message queues
- microservices

approach

- study after building systems
 - practical experience first
 - theory will make sense
 - case studies of real systems
-

extension 5.3: compilers and parsers

when to learn

- after completing bundlers
- when interested in language implementation
- natural progression from AST work

core concepts

- lexical analysis
- syntax analysis
- semantic analysis
- intermediate representation
- code generation
- optimization passes

resources

- "crafting interpreters" book
- typescript compiler source
- babel source

APPENDIX: RESOURCES

documentation

- node.js official docs
- V8 developer docs
- libuv documentation
- esbuild documentation
- fastify documentation
- bun documentation
- JavaScriptCore webkit blog

books

- "the go programming language" (donovan, kernighan)
- "understanding the Linux kernel"
- "node.js design patterns" (casciaro)
- "crafting interpreters" (nystrom)

blogs and articles

- v8.dev/blog
- webkit.org/blog
- bun.sh/blog
- nodejs.org/en/blog
- fastify.io/blog

tools

- astexplorer.net (AST visualization)
- [autocannon](https://autocannon.github.io) (benchmarking)
- [clinic.js](https://clinicjs.org) (profiling)
- [flamegraph](https://flamegraph.dev) (performance visualization)
- Chrome DevTools

repositories to study

- [esbuild](https://github.com/evanw/esbuild)
- [webpack](https://github.com/webpack/webpack)
- [fastify](https://github.com/fastify/fastify)
- [node.js](https://github.com/nodejs/node)
- [bun](https://github.com/bun/bun)
- [v8](https://github.com/v8/v8)
- [libuv](https://github.com/libuv/libuv)

VERIFICATION SYSTEM

how to verify understanding

level 1: can explain

- write explanation in your own words
- teach to someone else
- create diagrams
- compare with alternatives

level 2: can read source

- understand 40%+ of codebase
- trace execution flow
- identify key algorithms
- understand design decisions

level 3: can build

- implement simplified version
- handle edge cases
- optimize performance
- write tests

level 4: can debug

- diagnose production issues
- performance profiling
- fix bugs in real systems
- contribute to open source

level 5: mastery

- design systems from scratch
- make architectural decisions
- mentor others
- publish research/writing

progress tracking

daily

- ☐ 1+ hour of focused work
- ☐ notes on what learned
- ☐ questions that arose
- ☐ plan for next session

weekly

- ☐ completed projects
- ☐ blog post drafted/published
- ☐ review previous week
- ☐ plan upcoming week

monthly

- ☐ major milestone completed
- ☐ comprehensive blog series
- ☐ portfolio updated
- ☐ skills assessment

STUDY PRINCIPLES

how to learn effectively

1. read → build → teach

- read documentation and source code
- implement what you learned
- write about it publicly
- teaching solidifies understanding

2. depth over breadth

- understand one thing completely
- don't rush to next topic
- confusion means you're learning
- return to difficult concepts multiple times

3. source code is truth

- documentation can be outdated
- tutorials skip complexity
- real code shows real decisions
- read production codebases

4. build alongside reading

- every 2 days reading = 1 day building
- theory without practice is shallow
- projects reveal understanding gaps
- mistakes teach better than lectures

5. progressive difficulty

- start with simple concepts
- build complexity gradually
- each phase prepares for next
- don't skip fundamentals

6. public learning

- blog about what you learn
- share code on github
- engage in discussions
- accountability and portfolio

7. measure understanding

- can you explain it clearly?
- can you implement it?
- can you debug it?
- can you teach it?

common mistakes

avoid these:

- watching videos instead of reading code
- collecting resources without actually studying
- moving to next topic too quickly
- perfectionism blocking progress
- learning without building
- studying alone without sharing
- comparing your beginning with others' middle

instead:

- read source code and documentation
 - focus on one resource at a time
 - stay on topic until checkpoint cleared
 - ship imperfect projects
 - build as you learn
 - write publicly about progress
 - focus on your own growth
-

GETTING STARTED

first steps (right now)

1. set up environment

```
bash

# install tools
node --version # verify
go version    # install if needed
git --version # verify
code --version # or your editor
```

2. clone repositories

bash

`git clone https://github.com/evanw/esbuild`

`git clone https://github.com/fastify/fastify`

`git clone https://github.com/nodejs/node`

`git clone https://github.com/oven-sh/bun`

3. start with phase 1.1

- ☐ read survivejs "what is webpack" chapter
- ☐ read node module resolution docs
- ☐ write 200 words on why bundlers exist
- ☐ post for review

4. commit to process

- ☐ 1 hour minimum daily
- ☐ check in every 2-3 days
- ☐ complete verification checkpoints
- ☐ publish blog posts

FINAL NOTES

why this sequence works

bundlers first:

- closest to your current work
- immediate practical value
- introduces parsing, AST, module systems
- prepares for runtime internals

fastify second:

- leverages your migration experience
- introduces performance optimization
- connects to node.js internals
- practical server-side knowledge

node internals third:

- requires bundler and fastify context
- deepest technical level
- understanding event loop, V8, libuv
- foundation for bun comparison

bun last:

- requires node internals knowledge
- comparative understanding
- alternative architectures
- cutting-edge technology

this is not a race

- no timelines means no pressure
- depth matters more than speed
- understanding beats completion
- mastery takes time

you're ready

- you have the roadmap
- you have the resources
- you have the motivation
- you have the capability

start with phase 1.1 tomorrow.

read those 3 resources.

write 200 words.

post here for review.

that's day 1.

everything else follows from that first step.

end of study plan.